Noções de Complexidade de Algoritmos

Professor: André Felipe Ribeiro Cordeiro
Disciplina: Estruturas de Dados

Departamento de Informática
Centro de Tecnologia
Universidade Estadual de Maringá

Introdução

O que é problema?

- **Possível resposta**
  - Uma tarefa a ser executada.

- **Outra questão**
  - Todos os problemas podem ser solucionados por um ou mais computadores?

Introdução

O que é algoritmo?

- **Possível resposta**
  - “Sequência **finita** de instruções **computáveis**, sem ambiguidade, que transformam um **conjunto** de **valores iniciais** e satisfazem **condições específicas** de um problema”.

Introdução

Algoritmos *versus* Programas

- Programas são escritos com notação específica para que computadores realizem o processamento:
  - Pascal, C, C++, Java, PHP, etc...

- Algoritmos são escritos para que humanos os entendam, mas que possam ser codificados em alguma **linguagem de programação**...

Introdução

- Podemos estudar o comportamento de **algoritmos** e não de **programas**.

- Para saber se um algoritmo está correto, utiliza-se uma ou mais **técnicas matemáticas**.

- Para saber se um programa está correto, utiliza-se **testes**, tal como o **teste de mesa**.

Como avaliar algoritmos

- Podemos usar **três critérios** para **avaliar um algoritmo**.
  - Simplicidade
  - Corretude
  - Eficiência

Como avaliar algoritmos

- Podemos usar **três critérios** para **avaliar um algoritmo**.
  - **Simplicidade**
  - Corretude
  - Eficiência

Como avaliar algoritmos

- **Simplicidade**
  - Um algoritmo é simples se puder ser facilmente **entendido**, **implementado** e **mantido**.
  - Não se conhece **técnicas formais** para isto!

Como avaliar algoritmos

- Podemos usar **três critérios** para **avaliar um algoritmo**.
  - Simplicidade
  - **Corretude**
  - Eficiência

Como avaliar algoritmos

- **Corretude**
  - Um algoritmo está correto se para **toda entrada** específica a **saída correta é produzida**.
  - "*Testes servem apenas para provar que um algoritmo tem erros, nunca para provar que está correto*" *(Dijkstra).*

Como avaliar algoritmos

- Podemos usar **três critérios** para **avaliar um algoritmo**.
  - Simplicidade
  - Corretude
  - **Eficiência**

Como avaliar algoritmos

- **Eficiência**
  - A eficiência de um algoritmo está relacionada com uma medida quantitativa definida como a inversa da quantidade de ***recursos requeridos para o seu funcionamento***.

- **Que recursos são estes?**
  - Tempo, Memória e outros...

Métodos de Avaliação

- **Possíveis métodos de avaliação**
  - Analítico
  - Experimental

- Tais métodos podem ser claramente **diferenciados**.

Métodos de Avaliação

- **Possíveis métodos de avaliação**
  - **Analítico**
  - Experimental

- Tais métodos podem ser claramente **diferenciados**.

Métodos de Avaliação

❑ Método Analítico

- ❖ Construção de um **modelo matemático** para determinado algoritmo.

- ❖ A **comparação de eficiência** entre algoritmos se dá a partir da **comparação de modelos**.

Métodos de Avaliação

- **Possíveis métodos de avaliação**
  - Analítico
  - **Experimental**

- Tais métodos podem ser claramente **diferenciados**.

Métodos de Avaliação

- **Método Experimental**
  - Implementação de **diferentes algoritmos**.
  - Execução dos algoritmos com **entradas bem extensas**.
  - **Análise Estatística** dos Resultados.

Observações

- O **método analítico** é utilizado com **maior frequência**.

- O **método experimental** é utilizado em **último caso**, quando **não se obtém uma resposta** pelo método analítico.

Como medir a eficiência

- Para medir a eficiência, é necessário:
  - **Prever** a quantidade de recursos
    - ✓ Memória
    - ✓ Tempo de execução
  - Escolher um **modelo de computação** adotado para a sua **implementação**.

Observações

- A **memória** está associada com a **complexidade de espaço**.

- O **tempo de execução** está associado com a **complexidade de tempo**.

Como medir a eficiência

- **Modelo de Computação**
  - Ao invés de escolher uma **máquina particular** para avaliar a **eficiência dos algoritmos**, é mais conveniente selecionar um **modelo matemático de computador**.

Como medir a eficiência

- **Possível Modelo de Computação**

1. As operações são executadas sequencialmente.

2. A execução de toda e qualquer operação toma uma unidade de tempo.

3. A memória é infinita.

Complexidade de Tempo

- **Casos particulares mais importantes**:
  - Melhor caso
  - Pior caso
  - Caso médio

Complexidade de Tempo

- **Melhor caso**
  - O menor tempo de execução possível.
- **Pior caso**
  - Tempo máximo de execução.
- **Caso médio**
  - Tempo médio de execução para todo tipo de entrada possível.

Observação

- No caso médio, é necessário conhecer a **distribuição estatística** dos **dados de entrada**.

Exemplo
Complexidade de Tempo

- **BubbleSort**
  - Um dos métodos de ordenação mais conhecidos.

- **Ideia**
  - Percorrer um vetor de **n posições** por **n vezes**, a cada vez comparando dois elementos e trocando-os caso o primeiro seja maior que o segundo.

Exemplo
Complexidade de Tempo

```
bubbleSort(int vetor[], int tamanho)
    int i, j
    for i ← 0 to i ← (tamanho-2)
        for j ← 1 to j ← (tamanho-1)
            if vetor[j-1] > vetor[j]
                int temp ← vetor[j-1]
                vetor[j-1] ← vetor[j]
                vetor[j] ← temp
```

Exemplo
Complexidade de Tempo

- **Pior caso** para o BubbleSort (vetor em ordem decrescente)

	5	4	3	2	1	TROCAS
i=0, j=1	4	5	3	2	1	4
i=0, j=2	4	3	5	2	1	
i=0, j=3	4	3	2	5	1	
i=0, j=4	4	3	2	1	5	
i=1, j=1	3	4	2	1	5	3
i=1, j=2	3	2	4	1	5	
i=1, j=3	3	2	1	4	5	
i=2, j=1	2	3	1	4	5	2
i=2, j=2	2	1	3	4	5	
i=3, j=1	1	2	3	4	5	1

Exemplo
Complexidade de Tempo

- No caso de um vetor em ordem decrescente (**pior caso**), as **operações de troca** serão executadas

$$((n-1) + (n-2) + ... + 2 + 1) \text{ vezes}$$

- Aplicando a soma dos termos de uma Progressão Aritmética, tem-se

Soma = (Tam. da Seq. x (1º Termo + Último Termo)) / 2

Exemplo
Complexidade de Tempo

- Aplicando a soma dos termos de uma Progressão Aritmética, tem-se

Soma = (Tam. da Seq. x (1º Termo + Último Termo)) / 2

- Ou seja:

$$((n-1) \times n) / 2 = 0{,}5n^2 - 0{,}5n \text{ vezes}$$

Observações

- Consideremos **n** a variável que descreve o **tamanho da entrada de dados** do algoritmo.

- Consideremos **T(n)** a **função que descreve a eficiência** do algoritmo, em relação ao tamanho dos dados de entrada.

Exemplo
Complexidade de Tempo

- Se tomarmos as operações de **troca de valores** como critério para calcular a eficiência do algoritmo **BubbleSort**, podemos dizer que, para o **pior caso**:

$$T(n) = 0{,}5n^2 - 0{,}5n$$

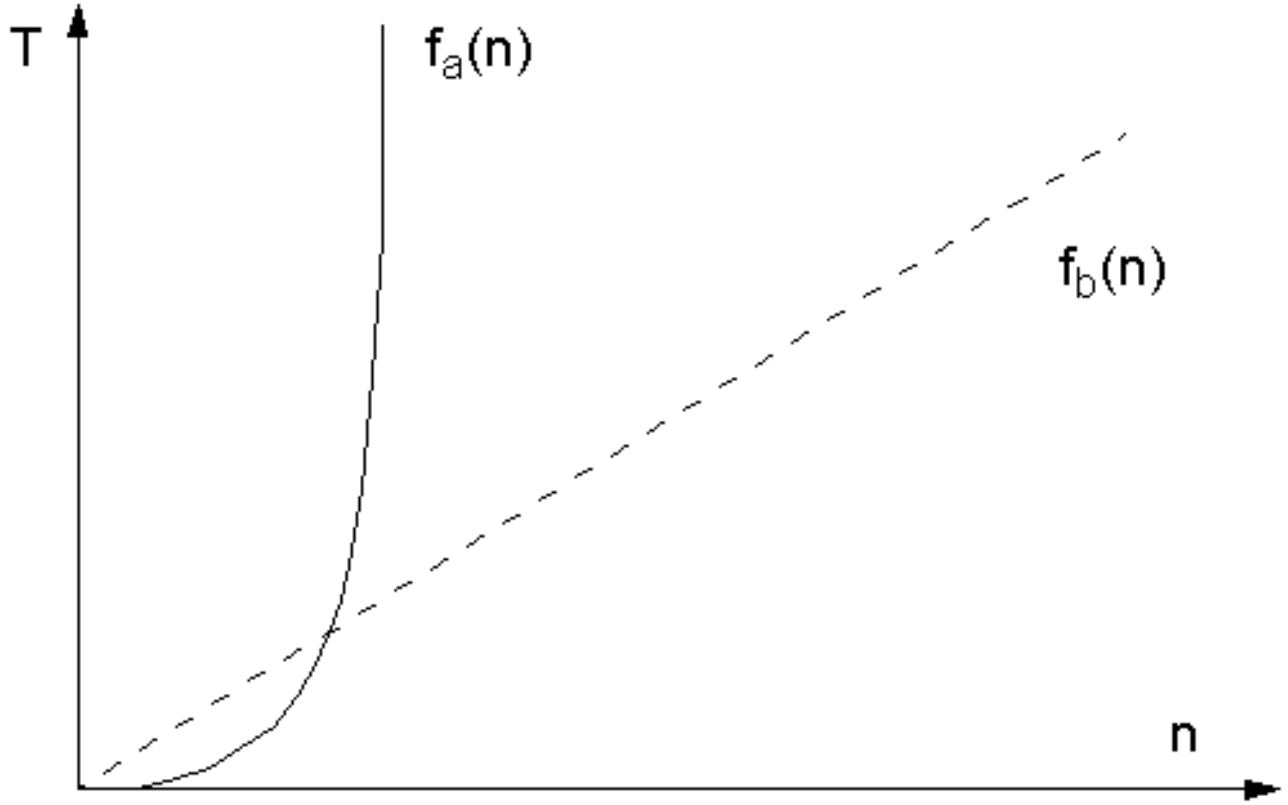
Observação

- Para **comparar dois algoritmos**, é necessário descrever seus **comportamentos**, em termos de **eficiência**, em função do **tamanho do conjunto** de dados de entrada.

Comparação de Algoritmos

- Vamos supor que temos dois algoritmos “a” e “b” para a solução de um problema.
- Se a complexidade de um é expressa por $T_a(n) = n^2$ e a do outro por $T_b(n) = 100n$.
- Isto significa que o algoritmo “a” cresce quadraticamente e que o algoritmo “b” cresce linearmente.

Comparação de Algoritmos



Comparação de Algoritmos

- A **comparação de eficiência de algoritmos** se dá por meio da comparação de **funções matemáticas**, que descrevem o **consumo de recursos** pelos algoritmos.

- As funções matemáticas descrevem o comportamento dos algoritmos para cada caso (**melhor**, **pior** e **médio**), em termos de **tempo de execução**.

Observações

- Para valores suficientemente pequenos de n, **qualquer algoritmo custa pouco** para ser executado, mesmo os algoritmos **ineficientes**.

- Por isso, a análise da complexidade de algoritmos é realizada para valores grandes de n.

Notações Assintóticas

- O comportamento assintótico de f(n) representa o **limite** do comportamento da "**função custo**" quando **n** cresce.

- É o **modo padrão** de indicar o tempo de execução de algoritmos.

- Se concentra no mais importante, ao abstrair os termos de **baixa ordem** e **fatores constantes**.

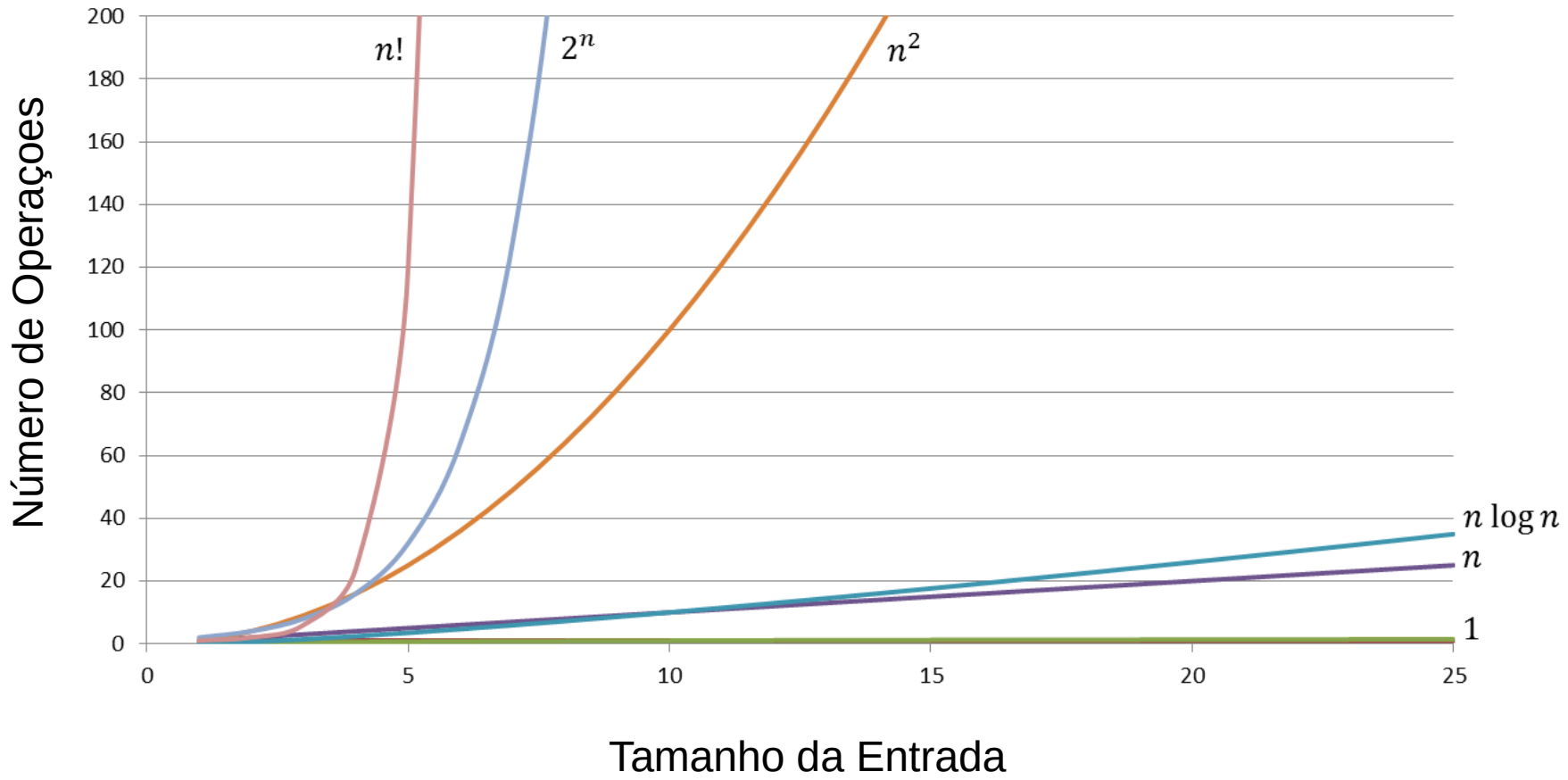
Notações Assintóticas

- Vamos **comparar funções assintoticamente**, para valores grandes, e desprezar constantes multiplicativas e termos de menor ordem.

	$n = 100$	$n = 1000$	$n = 10^4$	$n = 10^6$	$n = 10^9$
$\log n$	2	3	4	6	9
n	100	1000	10^4	10^6	10^9
$n \log n$	200	3000	$4 \cdot 10^4$	$6 \cdot 10^6$	$9 \cdot 10^9$
n^2	10^4	10^6	10^8	10^{12}	10^{18}
$100n^2 + 15n$	$1,0015 \cdot 10^6$	$1,00015 \cdot 10^8$	$\approx 10^{10}$	$\approx 10^{14}$	$\approx 10^{20}$
2^n	$\approx 1,26 \cdot 10^{30}$	$\approx 1,07 \cdot 10^{301}$	?	?	?

Notações Assintóticas

- Funções mais comuns



Notações Assintóticas

- A notação assintótica é utilizada para comparar funções com estes símbolos.

$$O \approx \leq$$

$$\Omega \approx \geq$$

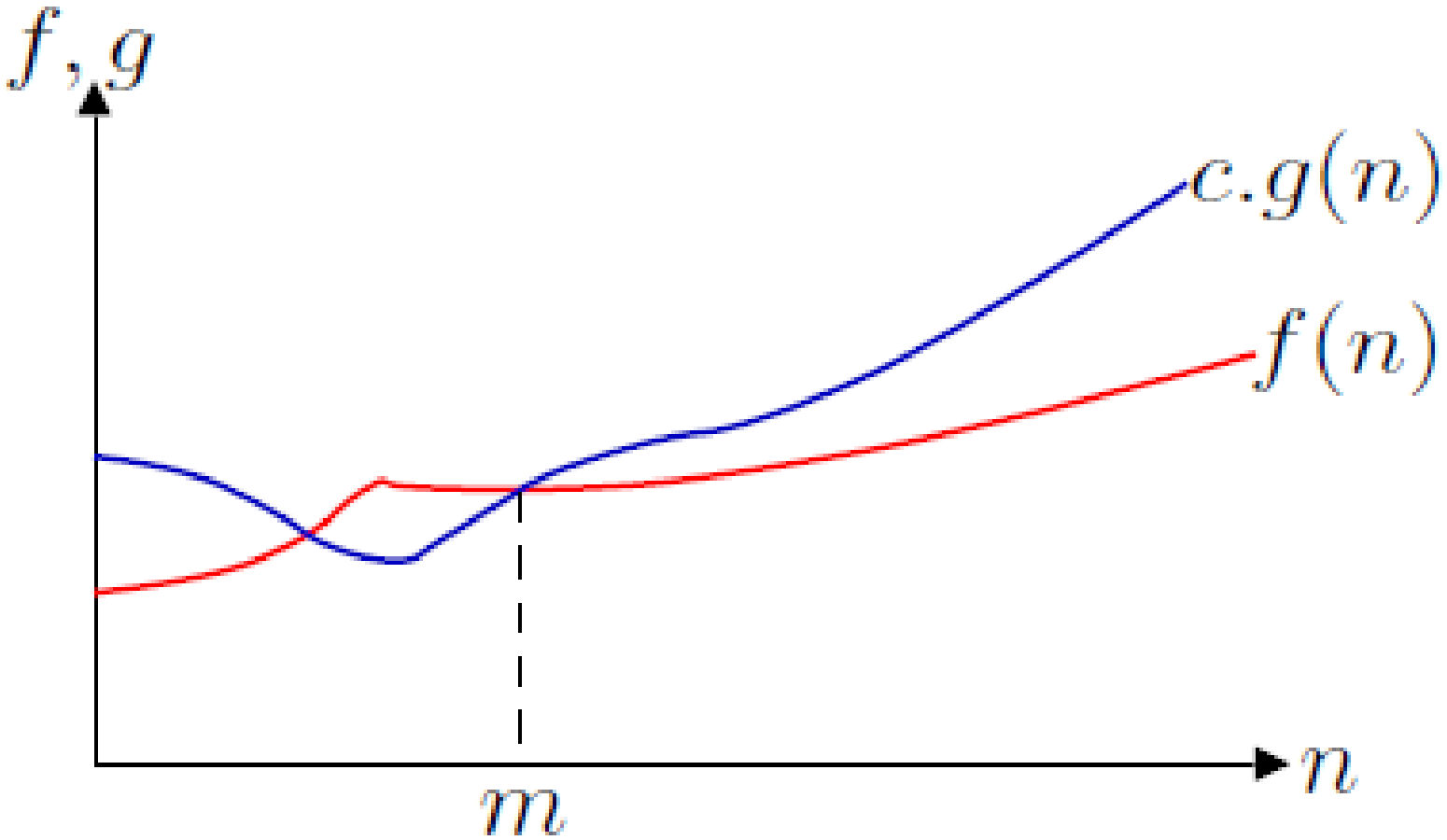
$$\Theta \approx =$$

Notação O

- Uma função f(n) é O(g(n)), ou **notação** f(n) = O(g(n)), se existem duas constantes positivas c e m tais que f (n) ≤ c.g(n), para todo n ≥ m.

- f (n) = O(g(n)) : g(n) é **limite superior** para f (n).

Notação O



Notação O

- **Exemplo 1**
  - Seja $f(n) = n^2 + 2n + 1$.
  - É verdade que $f(n) = O(n^2)$?

$$\mathbf{n^2 + 2n + 1 \leq n^2 + 2n^2 + n^2 = 4n^2}, \text{ sempre que } \boldsymbol{n \geq 1}$$

- Se considerarmos c = 4 e m = 1, verificaremos que é verdade!

Notação O

- **Exemplo 2**
  - Seja $f(n) = n^2$.
  - É verdade que $f(n) = O(n)$?
  - Existem c e m tal que $n^2 \leq cn$ para $n \geq m$?
  - E se dividirmos ambos os lados por n, cujo resultado é igual $n \leq c$?
  - Não!

Notação O

- **Exemplo 3**
  - $f(n) = 20n^3 + 10n\lg n + 1$ é $O(n^3)$?

- **Exemplo 4**
  - $f(n) = n^3$ é $O(1)$?

Notações Assintóticas

- A notação assintótica é utilizada para comparar funções com estes símbolos.

$$O \approx \leq$$

$$\Omega \approx \geq$$

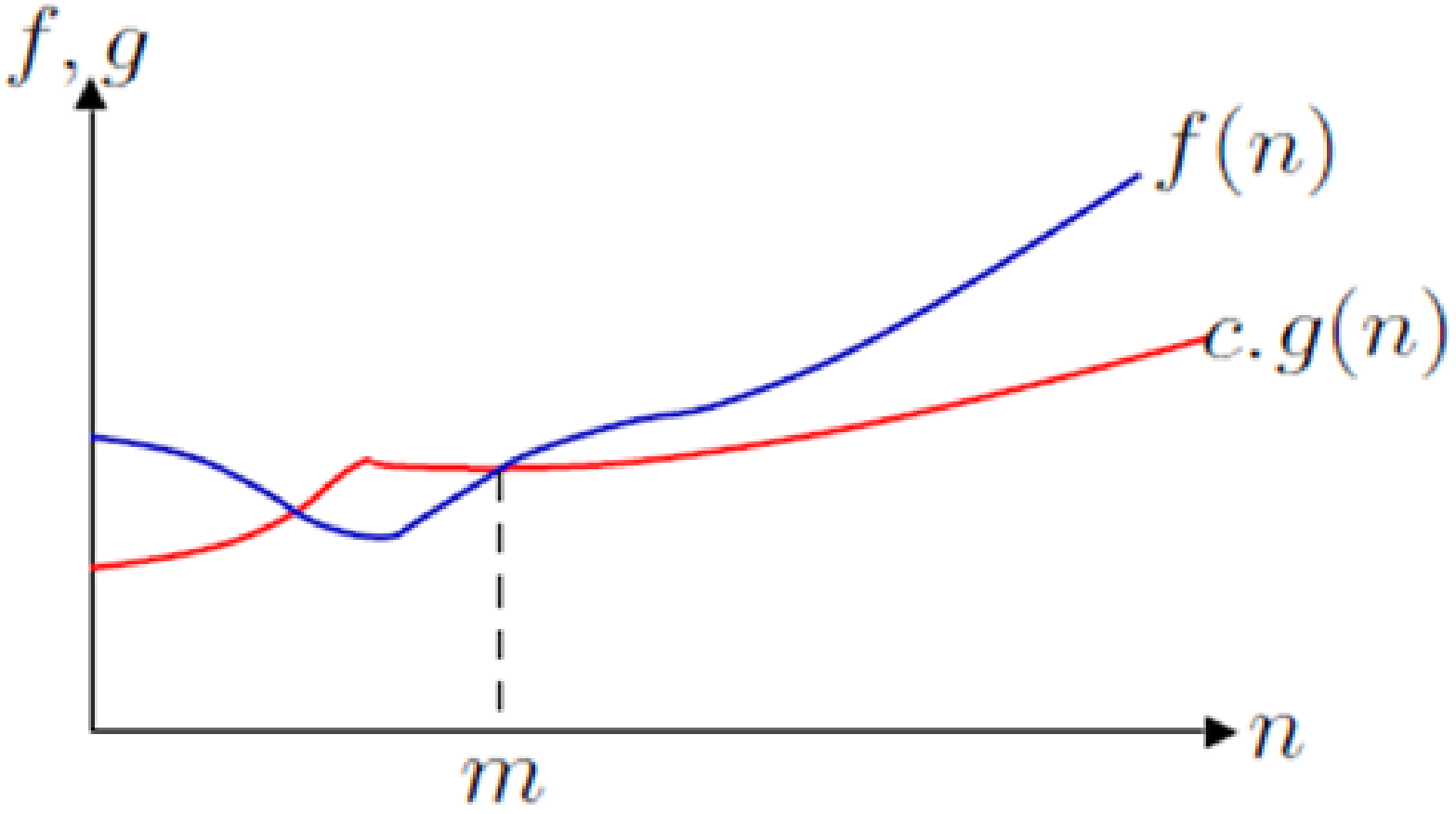
$$\Theta \approx =$$

Notação Ω

- Uma função f(n) é Ω(g(n)), **notação** f(n) = Ω(g(n)), se existem duas constantes positivas c e m tais que f(n) ≥ c.g(n), para todo n ≥ m.

- f(n) = Ω(g(n)) : g(n) é **limite inferior** para f(n). Tal limite está associado com o tempo mínimo do algoritmo.

Notação Ω



Notação Ω

- **Exemplo 5**
  - $f(n) = 20n^3$ é $\Omega(n^3)$?

- **Exemplo 6**
  - $f(n) = n^3$ é $\Omega(1)$?

Notações Assintóticas

- A notação assintótica é utilizada para comparar funções com estes símbolos.

$$O \approx \leq$$

$$\Omega \approx \geq$$

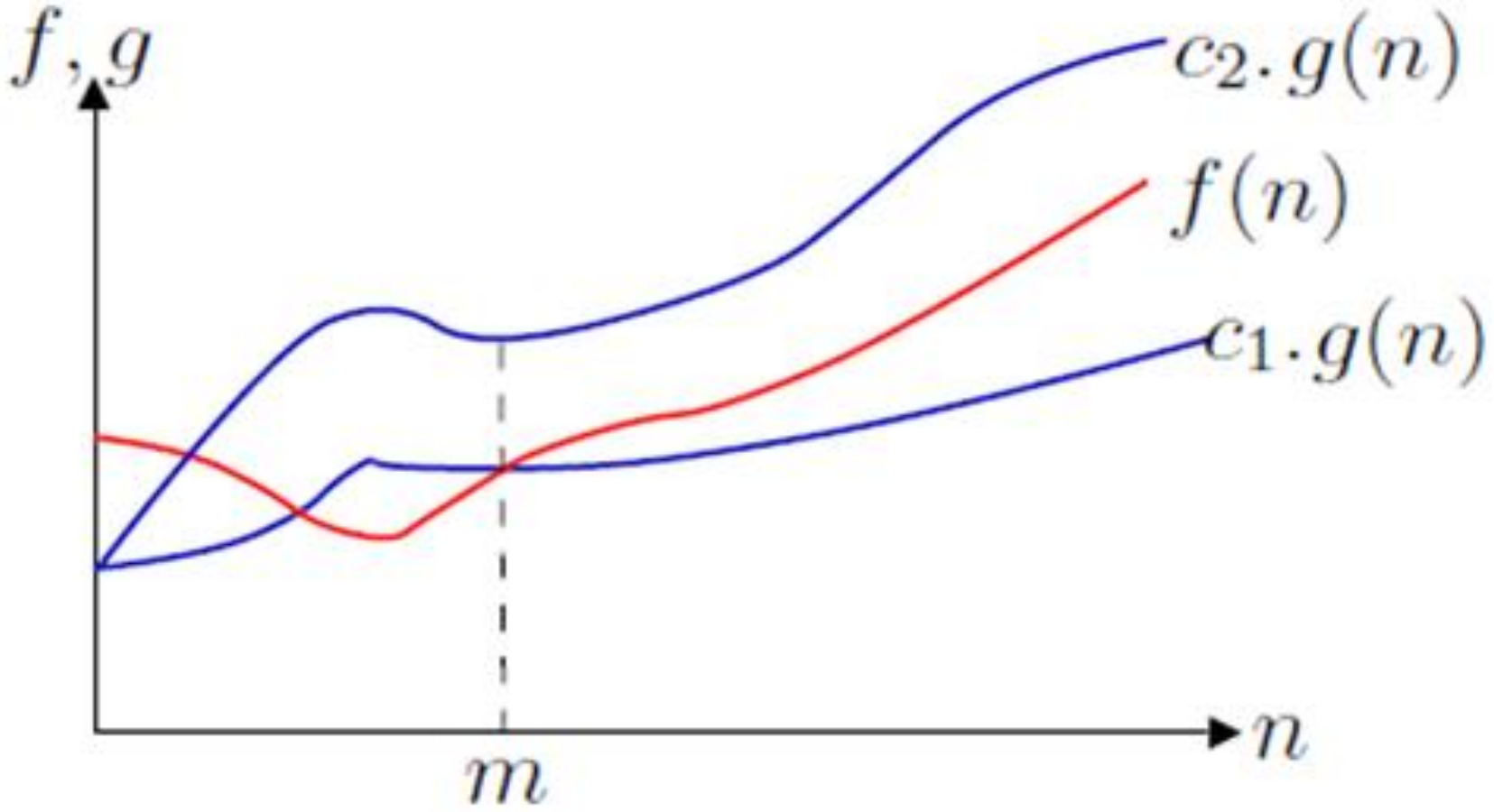
$$\Theta \approx =$$

Notação θ

- Uma função $f(n)$ é $\theta(g(n))$, **notação** $f(n) = \theta(g(n))$, se existem duas constantes positivas c_1, c_2 e m tais que $0 \leq c_1.g(n) \leq f(n) \leq c_2.g(n)$, para todo $n \geq m$.

- $f(n) = \theta(g(n))$: $g(n)$ é **limite restrito** para $f(n)$ ou limite assintótico firme.

Notação θ



Notação Ω

- **Exemplo 7**
  - $f(n) = 20n^3$ é $\theta(n^3)$?

Regras Importantes

- Para obter a **função de complexidade associada** com um **algoritmo**, é importante considerar **regras de cálculo**.

- As principais regras são apresentadas na sequência.

Regras Importantes

- **Regra 01:** $f(n) = O(f(n))$

- **Regra 02:** $O(f(n)) + O(f(n)) = O(f(n))$

Regras Importantes

- **Regra 03:** Regra da **soma** de tempos
  - Se $T1(n) = O(f(n))$ e $T2(n) = O(g(n))$ então $T1(n) + T2(n) = O(\mathbf{max}(f(n), g(n)))$
  - Isto significa que a complexidade de um algoritmo com **dois trechos** em sequência, com tempos de execução diferentes, é dada como a complexidade do trecho de **maior complexidade**.

Regras Importantes

- **Regra 04**: Regra do **produto** de tempos
  - Se T1(n) = O(f(n)) e T2(n) = O(g(n)) então T1(n) * T2(n) = O(f(n) * g(n))
  - Isto significa que a complexidade de um algoritmo com **dois trechos aninhados**, em que o segundo é repetidamente executado pelo primeiro, é dada como o **produto da complexidade** do trecho **mais interno** pela complexidade do trecho **mais externo**.

Observações

- Regras rígidas sobre o cálculo da complexidade de qualquer algoritmo **não existem**. Cada caso deve ser estudado **em suas condições**.

- No entanto, as **estruturas de controle clássicas da programação estruturada** permitem uma estimativa típica.

Observação

- Algoritmos construídos a partir de **combinações** dessas **estruturas de controle** podem ter **sua complexidade mais facilmente estimada**.

Complexidades de Estruturas de Controle

1. Comando simples

- Possui um tempo de execução **constante**
- $\theta(c) = \theta(1)$

Complexidades de Estruturas de Controle

2. Sequência de Comandos

- Possui um tempo de execução igual à **soma** dos tempos de cada comando da sequência.
- Se cada comando é $\theta(1)$, então também será a sequência.
- Senão, **pela regra da soma**, a sequência terá a complexidade do comando de maior complexidade.

Complexidades de Estruturas de Controle

3. Alternativa

- Qualquer um dos ramos pode apresentar uma complexidade arbitrária.
- A complexidade resultante é a **maior delas**.
- Isto vale para alternativa dupla (*if-else*) ou múltipla (*switch*).

Complexidades de Estruturas de Controle

4. Repetições

- ***Repetição Contada***: é aquela em que cada iteração atualiza o controle mediante uma **adição**.
- Se o número de iterações é **independente do tamanho do problema**, a complexidade de toda a repetição é a **complexidade do corpo da mesma**, pela **regra da constante** ou pela **regra da soma de tempos**.

Complexidades de Estruturas de Controle

4. Repetições

❖ ***Repetição Contada*** (Exemplos)

```
for(i = 0; i < k; i++)                    /* se k não é f(n) então
  Trecho com complexidade O(g(n))            o trecho é O(g(n)) */

for(i = 0; i < 10; i++)                   /* isto é O(1), logo toda
  x = x + v;                                  a repetição é O(1) */
  printf("%d", x);
```

Complexidades de Estruturas de Controle

4. Repetições

- **Repetição Contada:** se o número de iterações é **em função de n**, pela regra do produto, tem-se a complexidade da repetição como a complexidade do corpo **multiplicada** pela função que descreve o **número de iterações**.

```
for(i = 0; i < n; i++)      /* como o nº de iterações é f(n) = n
   Trecho O(g(n))              então o trecho é O(n*g(n))  */
```

Complexidades de Estruturas de Controle

4. Repetições

- **Repetição Contada:** uma aplicação comum da regra do produto é a **determinação da complexidade de repetições aninhadas**.

```
for(i = 0; i < n; i++)
    for(j = 0; j < n; j++)
        Trecho com O(1)
```

/* **o trecho é O(f(n)*g(n)), no caso g(n) = n*1 (laço interno). Logo, O(n*n), ou seja: O(n^2)** */

Complexidades de Estruturas de Controle

4. Repetições

- **Repetição Contada*:*** na situação a seguir, a repetição interna "**começa onde a externa parou**".

```
for(i = 0; i < n; i++)
  for(j = i; j < n; j++)
        Trecho com O(1)
```

/* **o laço interno é executado n+n-1 +...+3+2+1 = n(n+1)/2 vezes, logo, O(n(n+1)/2), ou seja:**

$O(0.5n^2+0.5n)) = O(n^2)$ */

Complexidades de Estruturas de Controle

4. Repetições

- **Repetições Multiplicativa***:* É aquela em que cada iteração atualiza o controle mediante uma *multiplicação* ou *divisão*.

```
limite = 1;
while (limite <= n) {
   Trecho com O(1);
   limite = limite * 2;
}
```

/* **o número de iterações depende de n; limite vai dobrando a cada iteração; depois de k iterações, limite = 2^k e k = $\log_2$; como o valor máximo de limite é n, então o trecho é $O(\log_2 n) = O(\log n)$** */

Complexidades de Estruturas de Controle

5. Funções Recursivas

- Para funções recursivas, é importante observar o **número de vezes** que o procedimento é **invocado recursivamente**.
- Esse número é dependente do **tamanho do problema** e da **taxa de redução do tamanho do problema**, a cada **invocação da recursão**.
- É na determinação desse número que reside a **dificuldade** da **análise de algoritmos recursivos**.

Complexidades de Estruturas de Controle

5. Funções Recursivas

```
void mergeSort(int lista[], int inicio, int fim){
   if (inicio < fim){
      meio = (inicio + fim) / 2;
      mergeSort(lista, inicio, meio);
      mergeSort(lista, meio + 1, fim);
      merge(lista, inicio, fim, meio);
   }
}
```

- **E AGORA?** Utilizamos um método chamado **recorrência**!

Recorrências

- O custo do *MergeSort* é dado por uma recorrência que utiliza o conceito de **dividir pra conquistar**.

$$T(n) := \begin{cases} c; & se\ n = 1 \\ 2T\left(\frac{n}{2}\right) + cn; & se\ n > 1 \end{cases}$$

Considerações Finais

- Considere dois algoritmos A e B com tempo de execução $O(n^2)$ e $O(n^3)$, respectivamente.

- Qual deles é o mais eficiente ?

Considerações Finais

- **O(1) :** constante – mais rápido impossível!
- **O(log log n) :** super-rápido.
- **O(log n) :** logarítmico – muito bom.
- **O(n) :** linear – é o melhor que se pode esperar se algo não pode ser determinado **sem examinar toda a entrada**.

Considerações Finais

- **O(n log n) :** limite de muitos problemas práticos.
  - **Exemplo:** Ordenar uma coleção de números.
- **O(n^2) :** quadrático – ruim, comum para algoritmos simples
- **O(n^k) :** polinomial – ok, **apenas para n pequeno**
- **O(k^n), O (n!), O (n^n) :** exponencial – evitar ao máximo!

Referências

CORMEN, T. H.; LEISERSON, C. E.; STEIN, C. ***Algoritmos: Teoria e Prática***, 3ª ed., Editora Campus, 2012.